



BITS Pilani

Pilani | Dubai | Goa | Hyderabad



Embedded System Design
Dr. Meetha V Shenoy,
Dept of EEE, BITS Pilani Pilani Campus

PL resources –Zedboard Artix-7 FPGA technology

Meetha V Shenoy

Problem

- Implement a **3×3 convolution filter** on a **28×28 grayscale image**
- Input: 28×28 pixels (each pixel = 8 bits)
- Kernel: 3×3 filter coefficients
- Output: 26×26 pixels (after convolution)
- For a 3×3 kernel, 9 multiplications + 8 additions per output pixel.

Storage of Image

- Full image size = $28 \times 28 \times 8$ bits = 6,272 bits $\approx$ 0.77 KB — tiny.
- One BRAM (36 Kb) can store many such images

$$W = \begin{bmatrix} w_{00} & w_{01} & w_{02} \\ w_{10} & w_{11} & w_{12} \\ w_{20} & w_{21} & w_{22} \end{bmatrix} \quad X = \begin{bmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{bmatrix}$$

$$\begin{aligned} y &= w_{00}x_{00} + w_{01}x_{01} + w_{02}x_{02} \\ &\quad + w_{10}x_{10} + w_{11}x_{11} + w_{12}x_{12} \\ &\quad + w_{20}x_{20} + w_{21}x_{21} + w_{22}x_{22} \end{aligned}$$

$$\begin{aligned} p_0 &= w_{00}x_{00} + w_{01}x_{01} + w_{02}x_{02} \\ p_1 &= w_{10}x_{10} + w_{11}x_{11} + w_{12}x_{12} \\ p_2 &= w_{20}x_{20} + w_{21}x_{21} + w_{22}x_{22} \end{aligned}$$

$$y = p_0 + p_1 + p_2$$

Multiplication
 Partial sum (p0,p1,p2)
 Final sum

Pipelining

Stage	Operation	PL element used
Stage 1	Fetch 3×3 window pixels	BRAM (line buffers) /Distributed RAM(Coefficients)
Stage 2	Multiply: pixel × kernel coefficient (9 parallel products)	DSP48 (or LUT-based mul)
Stage 3	Local/row partial sums (e.g., sum of each row: 3 products → p0,p1,p2)	LUT + carry-chain (CARRY4) — pipelined registers after this stage
Stage 4	Final accumulation: add the partial sums → final convolution output y	LUT + carry-chain (CARRY4) — pipelined register at output
Stage 5	Activation / clipping / normalization (ReLU, scaling, requant)	LUT / small DSP / shift logic
Stage 6	Write/store result to memory	BRAM / distributed RAM / AXI streaming out

Without pipelining, the latency = sum of all stages.

With pipelining, each stage works on a different pixel simultaneously, boosting throughput.

Memory Architecture (BRAM + Distributed RAM)

- To apply a 3×3 window, you need 3 consecutive image rows available at once.
- Each row = 28 pixels = $28 \times 8 = 224$ bits
- 3 rows = 672 bits

Type	Why?	Where
Block RAM (BRAM)	Stores full image lines efficiently	Used for line buffers
Distributed RAM (in SliceM)	For coefficient storage	Used for kernel coefficients

BRAM = high density, larger capacity

Distributed RAM = close to logic, faster access for small storage

Shift Registers for Pixel Windows

- To shift pixels through the 3×3 window each clock cycle:
- Use **Shift Register LUTs (SRL32)** inside **SliceM**.
- Each SRL holds a small FIFO (e.g., last 3 pixels in a row).
- → This avoids using BRAM for tiny buffers.

8 bit addition??

- Sum[3:0] → First CARRY4 (handles bits 0–3)
- Sum[7:4] → Second CARRY4 (handles bits 4–7)

- $SUM[i] = A[i] \text{ XOR } B[i] \text{ XOR } CIN$
- $COUT = (A[i] \text{ AND } B[i]) \text{ OR } (CIN \text{ AND } (A[i] \text{ XOR } B[i]))$

Step 5: Storage and Output

- Final result (26×26) can be written back to BRAM (for larger image maps).

Function	Typical FPGA Resource	Reason
Line buffers	BRAM	Large sequential storage
Kernel coefficients	Distributed RAM / Registers	Small, static storage
Pixel window shift registers	SRL32 in SliceM	Fast, low-latency
Multiplication	DSP48E1	Dedicated multiplier-accumulator
Addition (adder tree)	Carry chain (CARRY4) in SliceL/M	Fast summation
Output buffer	BRAM / distributed RAM	Stores processed pixels

Slice Type	Capability	Benefit in This Design
SliceM	Supports LUTRAM & SRL	For small FIFOs (shift registers) and kernel memory
SliceL	Logic-only slice (with carry chain)	For arithmetic logic like adders
Carry Chain (CARRY4)	In both SliceL and SliceM	Enables fast adder tree (low delay)